



# Analysis & Development of Algorithms

V

Anton Kovantsev

[https://t.me/pop\\_antoniij](https://t.me/pop_antoniij)

# Graphs and their applications

An (undirected) graph is a pair  $G = (V, E)$ , where  $V$  is a set whose elements are called vertices (or nodes), and  $E$  is a set of two-sets (sets with two distinct elements) of vertices, whose elements are called edges (or links).

The number of vertices is usually denoted by  $|V|$ . The number of edges is usually denoted by  $|E|$ .

A directed graph is a graph in which edges have orientations.

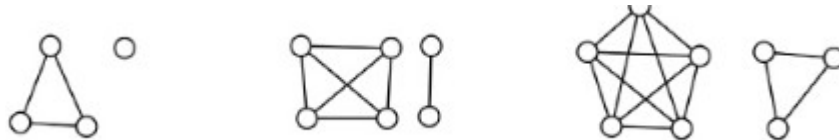
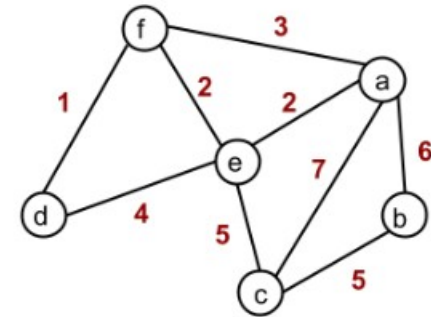
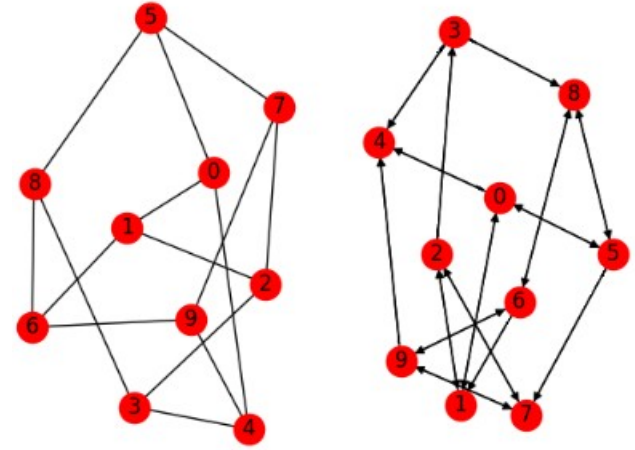
A weighted graph is a graph in which a weight is assigned to each edge.

A simple graph allows only one edge between a pair of vertices.

A multigraph is a generalization that allows multiple edges between a pair of vertices.

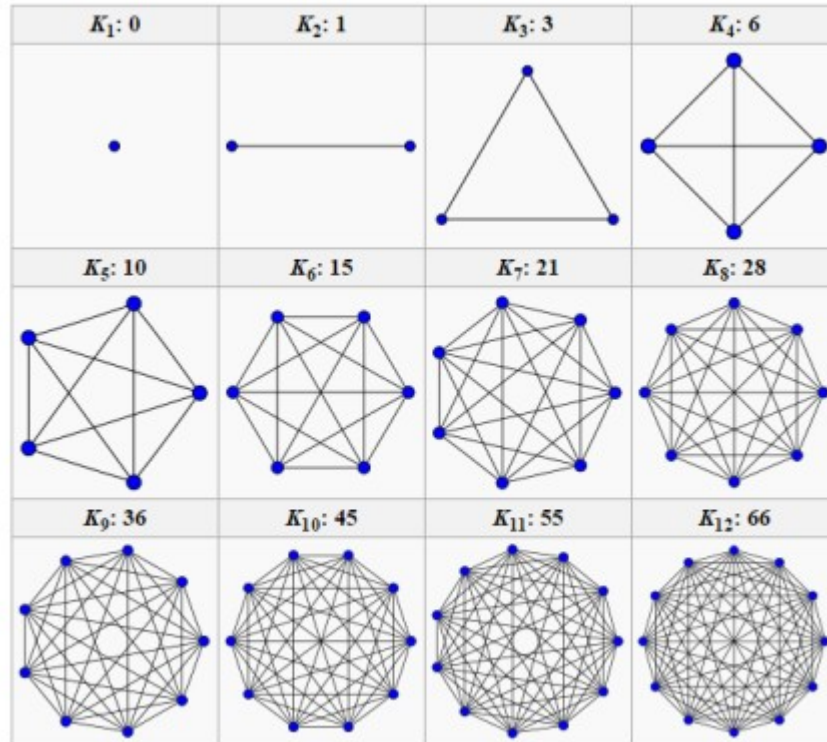
A pair of vertices  $(v_1, v_2)$  is called connected if there is a path from  $v_1$  to  $v_2$ . Otherwise, a pair of vertices is called disconnected.

A connected graph is one in which every pair of vertices is connected. Otherwise, it is a disconnected one.



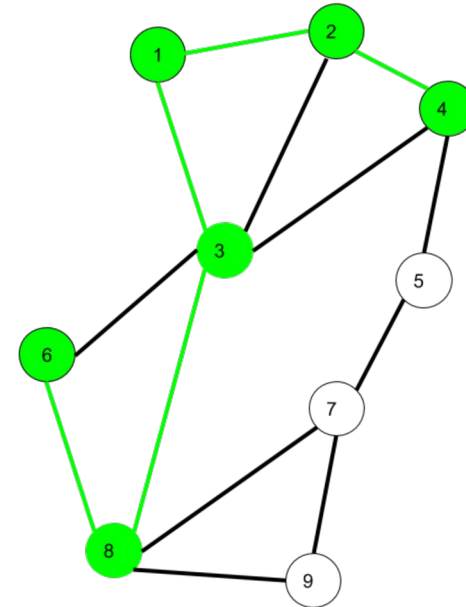
# Graphs and their applications

A complete graph is a graph in which each pair of vertices is joined by an edge.



A path in a graph is a sequence of distinct edges which join a sequence of distinct vertices. The length of a path is the sum of the weights of the traversed edges.

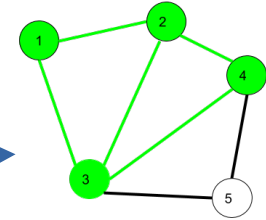
6->8->3->1->2->4 is a Path



# Graphs and their applications

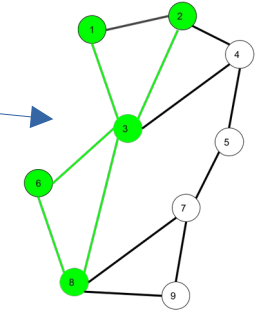
A walk is a sequence of vertices and edges of a graph i.e. if we traverse a graph then we get a walk. Edge and Vertices can both be repeated.

1->2->3->4->2->1->3 is a **walk**.



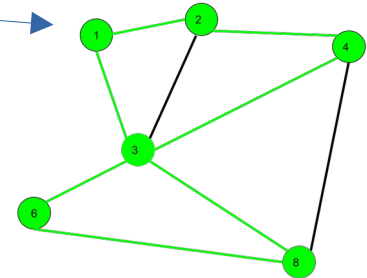
Trail is an open walk in which no edge is repeated, and vertex can be repeated.

1->3->8->6->3->2 is trail; Also 1->3->8->6->3->2->1 will be a closed **trail**



Circuit: Vertex can be repeated. Edges cannot be repeated.

1->2->4->3->6->8->3->1 is a **circuit**.

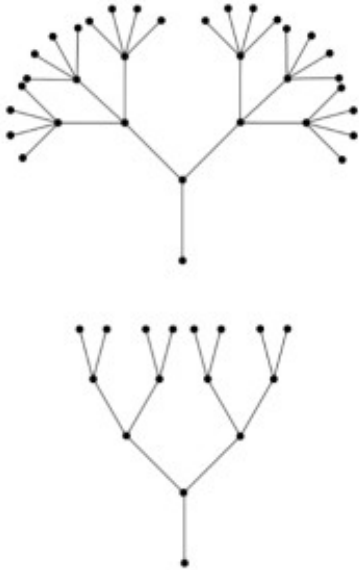


Path is a trail in which neither vertices nor edges are repeated i.e. if we traverse a graph such that we do not repeat a vertex and nor we repeat an edge.

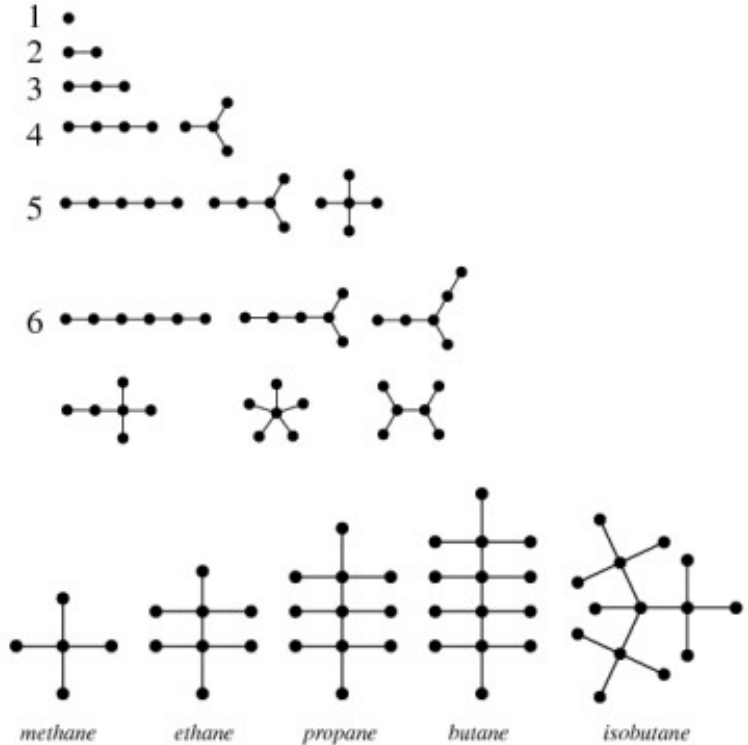
6->8->3->1->2->4 is a Path

# Special types of graphs: trees

A **tree** is an undirected graph in which any two vertices are connected by exactly one path.

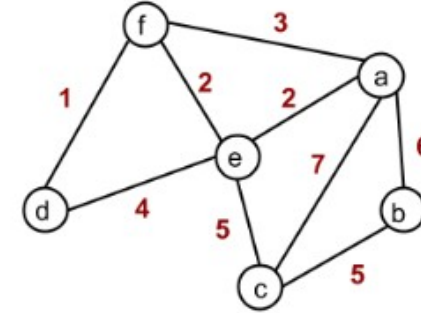
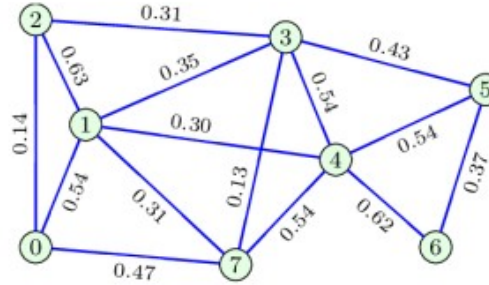
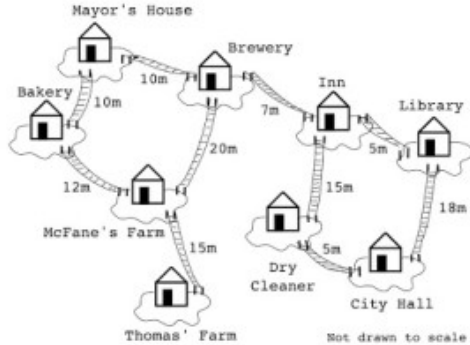


A **forest** is an undirected graph in which any two vertices are connected by at most one path, or equivalently a disjoint union of trees.

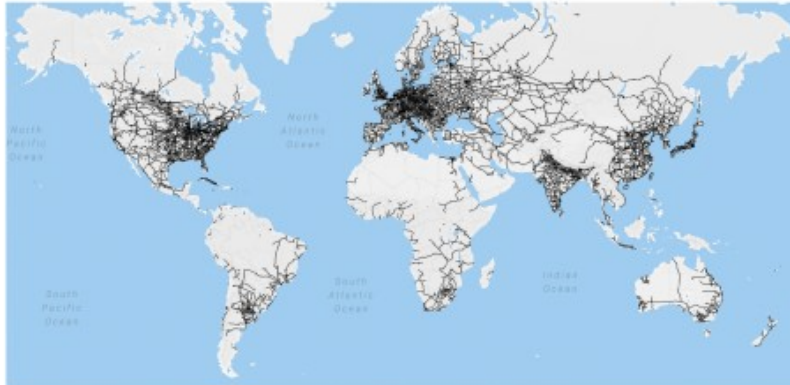


# Weighted graph

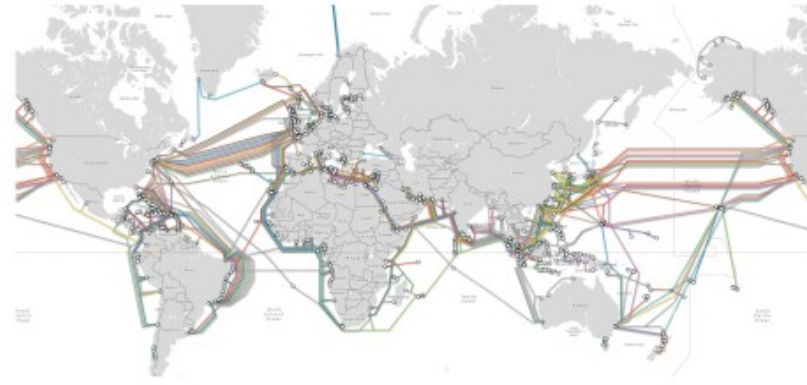
A **weighted graph** is a graph in which a weight is assigned to each edge.



Railway network



Undersea Cable network



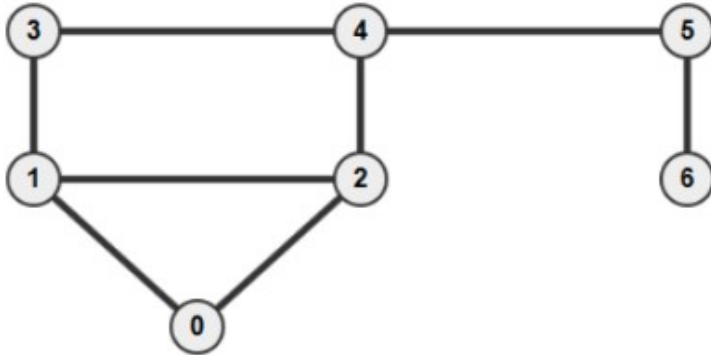
# Graph representations

## adjacency matrix and adjacency list

The adjacency matrix is a matrix whose rows and columns are indexed by vertices and whose cells contain a Boolean value that indicates whether the corresponding vertices are adjacent (for weighted graphs, it contains corresponding weights instead of 1s.). The matrix (stored as a 2D array) requires  $O(|V|^2)$  of space.

The adjacency list is a collection of lists containing the set of adjacent vertices of a vertex. The list (stored as an 1D array of lists) requires  $O(|V| + |E|)$  of space. For a sparse graph, i.e. a graph in which most pairs of vertices are not connected by edges,  $|E| \ll |V|^2$ , an adjacency list is significantly more space-efficient than an adjacency matrix.

Graph



Adjacency matrix

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 | 0 | 0 | 0 |
| 2 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 3 | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 4 | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| 5 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |
| 6 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |

Adjacency list

| Vertex | Adjacent vertices |
|--------|-------------------|
| 0      | 1, 2              |
| 1      | 0, 2, 3           |
| 2      | 0, 1, 4           |
| 3      | 1, 4              |
| 4      | 2, 3, 5           |
| 5      | 4, 6              |
| 6      | 5                 |

# Real world graphs (networks)

Link 1: Stanford Large Network Dataset Collection <https://snap.stanford.edu/data/>

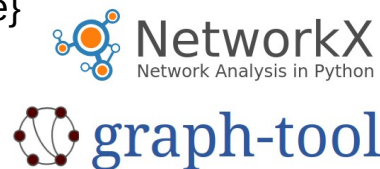
Link 2: Network Repository <https://networkrepository.com/>

## Stanford Large Network Dataset Collection

- **Social networks** : online social networks, edges represent interactions between people
- **Networks with ground-truth communities** : ground-truth network communities in social and information networks
- **Communication networks** : email communication networks with edges representing communication
- **Citation networks** : nodes represent papers, edges represent citations
- **Collaboration networks** : nodes represent scientists, edges represent collaborations (co-authoring a paper)
- **Web graphs** : nodes represent webpages and edges are hyperlinks
- **Amazon networks** : nodes represent products and edges link commonly co-purchased products
- **Internet networks** : nodes represent computers and edges communication
- **Road networks** : nodes represent intersections and edges roads connecting the intersections
- **Autonomous systems** : graphs of the internet
- **Signed networks** : networks with positive and negative edges (friend/foe, trust/distrust)
- **Location-based online social networks** : social networks with geographic check-ins
- **Wikipedia networks, articles, and metadata** : talk, editing, voting, and article data from Wikipedia
- **Temporal networks** : networks where edges have timestamps
- **Twitter and Memetracker** : memetracker phrases, links and 467 million Tweets
- **Online communities** : data from online communities such as Reddit and Flickr

**NB:** For consistency, I recommend to distinguish {graph, vertex, edge} and {network, node, link}.

**BUT:** Networkx (<https://networkx.org/>) uses {node, edge}



Data & Network Collections. Find and interactively **VISUALIZE** and **EXPLORE** hundreds of network data

|                        |     |                         |      |                        |     |
|------------------------|-----|-------------------------|------|------------------------|-----|
| ANIMAL SOCIAL NETWORKS | 816 | INTERACTION NETWORKS    | 29   | SCIENTIFIC COMPUTING   | 11  |
| BIOLOGICAL NETWORKS    | 37  | INFRASTRUCTURE NETWORKS | 8    | SOCIAL NETWORKS        | 77  |
| BRAIN NETWORKS         | 116 | LABELED NETWORKS        | 105  | FACEBOOK NETWORKS      | 114 |
| COLLABORATION NETWORKS | 20  | MASSIVE NETWORK DATA    | 21   | TECHNOLOGICAL NETWORKS | 12  |
| CHEMINFORMATICS        | 646 | MISCELLANEOUS NETWORKS  | 2669 | WEB GRAPHS             | 36  |
| CITATION NETWORKS      | 4   | POWER NETWORKS          | 8    | DYNAMIC NETWORKS       | 115 |
| ECOLOGY NETWORKS       | 6   | PROXIMITY NETWORKS      | 13   | TEMPORAL REACHABILITY  | 38  |
| ECONOMIC NETWORKS      | 16  | GENERATED GRAPHS        | 221  | BHOSLIB                | 36  |
| EMAIL NETWORKS         | 6   | RECOMMENDATION NETWORKS | 36   | DIMACS                 | 78  |
| GRAPH 500              | 8   | ROAD NETWORKS           | 15   | DIMACS10               | 84  |
| HETEROGENEOUS NETWORKS | 15  | RETWEET NETWORKS        | 34   | NON-RELATIONAL ML DATA | 211 |

# Knowledge graph

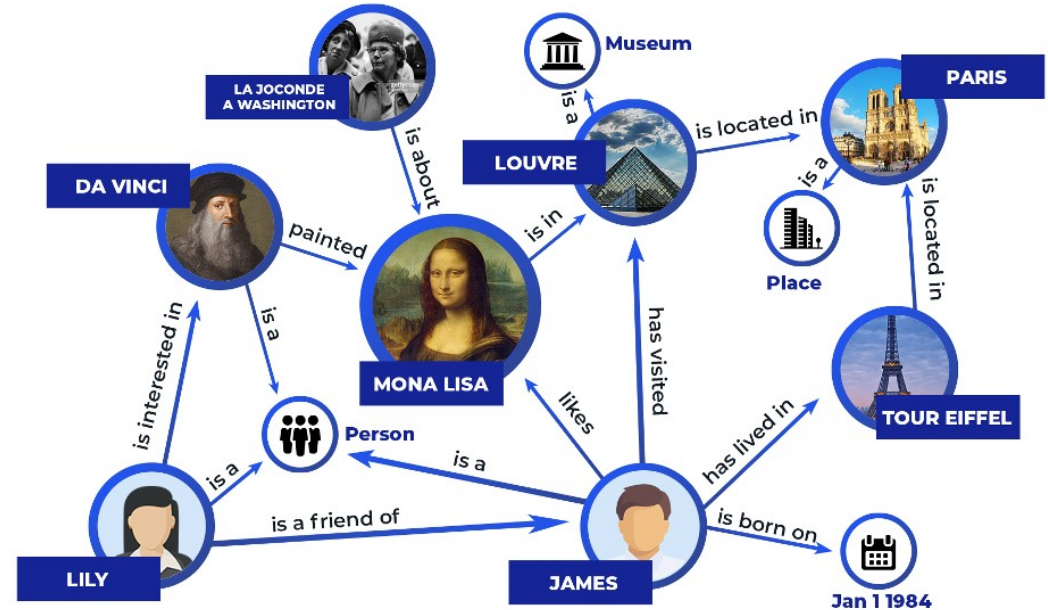
A knowledge graph is a structured data model that represents a network of real-world entities—such as people, places, concepts, or events—and illustrates the meaningful relationships between them.

**Nodes:** The individual data points or entities representing objects, places, or people (e.g., a person or a company).

**Edges:** The connecting lines that define how two nodes relate to one another (e.g., "works for" or "located in").

**Properties/Labels:** Detailed attributes attached to nodes and edges that add descriptive context and classification.

**Ontology:** The underlying framework or blueprint that defines the classes, rules, and categories for how information is structured.



# Depth-first search (DFS)

**Depth-first search (DFS)** is an algorithm for traversing a graph. The algorithm starts at a chosen root vertex and explores as far as possible along each branch before backtracking.

**Applied for:** searching connected components, searching loops in a graph, testing bipartiteness, topological sorting, etc.

The time complexity of DFS is  $O(|V|+|E|)$ .

A **connected component of a graph** is a subgraph in which any two vertices are connected by paths, and which is not connected to any vertex in the rest graph.

Follow the steps mentioned below to implement the idea using DFS:

Initialize all vertices as not visited.

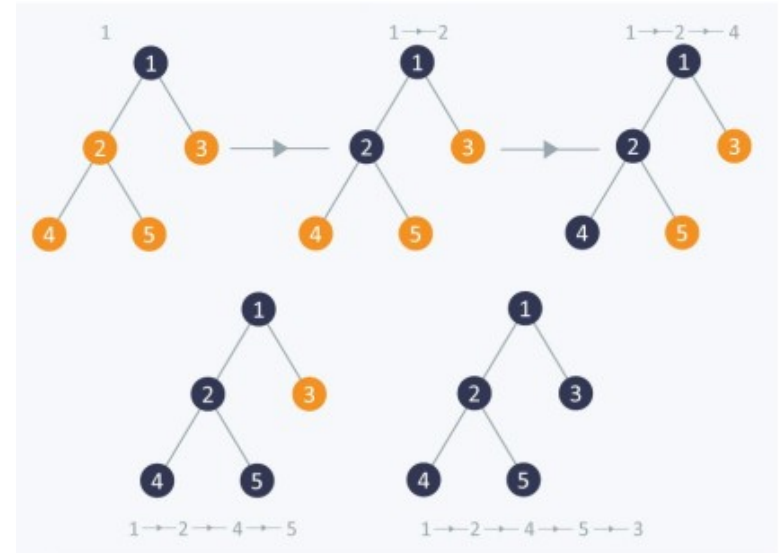
Do the following for every vertex  $v$ :

- If  $v$  is not visited before, call the DFS. and print the newline character to print each component in a new line

- Mark  $v$  as visited and print  $v$ .

- For every adjacent  $u$  of  $v$ ,

- If  $u$  is not visited, then recursively call the DFS.



# Breadth-first search (BFS)

**Breadth-first search (BFS)** is an algorithm for traversing or searching a graph. The algorithm starts at a chosen root vertex and explores all of the neighbour vertices at the present depth prior to moving on to the vertices at the next depth level.

It uses an opposite strategy to DFS, which instead explores the vertex branch as far as possible before being forced to backtrack and expand other vertices.

Applied for: searching shortest path.

The time complexity of BFS is  $O(|V| + |E|)$ .

**Initialization:** Enqueue the given source vertex into a queue and mark it as visited.

**Exploration:** While the queue is not empty:

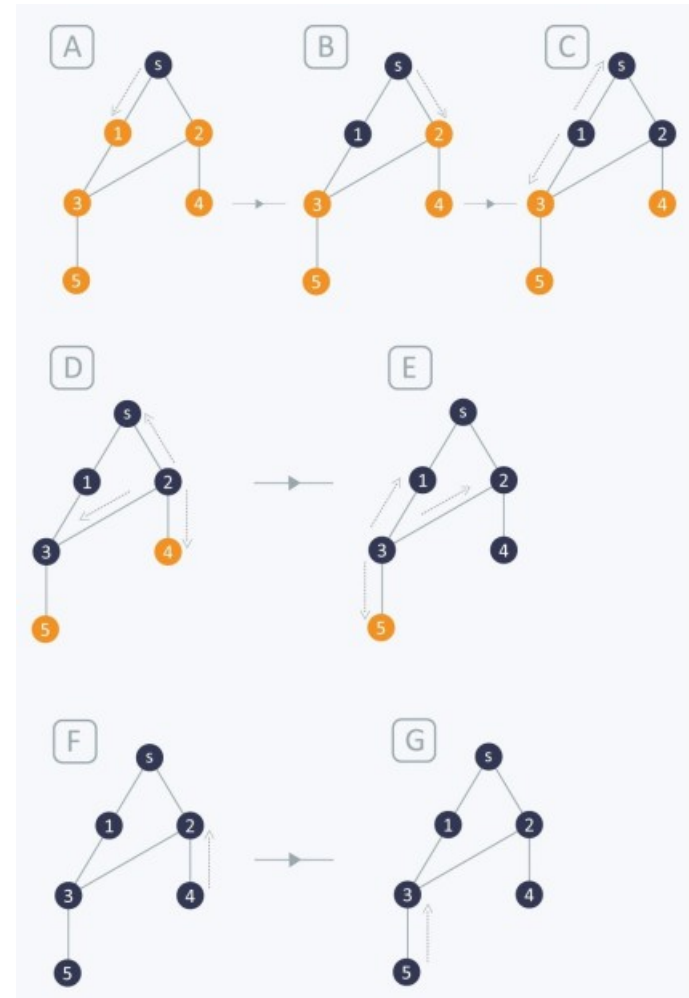
- Dequeue a node from the queue and visit it (e.g., print its value).

- For each unvisited neighbor of the dequeued node:

  - Enqueue the neighbor into the queue.

  - Mark the neighbor as visited.

**Termination:** Repeat step 2 until the queue is empty.



# Dijkstra's algorithm

**Problem:** given a weighted graph (with positive weights) and a source vertex, find shortest paths from the source to all other vertices.

**Main idea:** It generates a shortest path tree (SPT) with the source as a root, with maintaining two sets: one set contains vertices included in SPT, other set includes vertices not yet included in SPT. At every step, it finds a vertex which is in the other set and has a minimum distance from the source.

**Algorithm** (time complexity is from  $O(|V| \log |V|)$  to  $O(|V|^2)$  )

Create a set sptSet (shortest path tree set) that keeps track of vertices included in the shortest path tree, i.e., whose minimum distance from the source is calculated and finalized. Initially, this set is empty.

Assign a distance value to all vertices in the input graph. Initialize all distance values as INFINITE .

Assign the distance value as 0 for the source vertex so that it is picked first.

While sptSet doesn't include all vertices

    Pick a vertex  $u$  that is not there in sptSet and has a minimum distance value.

    Include  $u$  to sptSet .

    Then update the distance value of all adjacent vertices of  $u$  .

    To update the distance values, iterate through all adjacent vertices.

    For every adjacent vertex  $v$ ,

    if the sum of the distance value of  $u$  (from source) and weight of edge  $u-v$  , is less than the distance value of  $v$  , then update the distance value of  $v$  .

**Note:** We use a boolean array sptSet[ ] to represent the set of vertices included in SPT . If a value sptSet[ $v$ ] is true, then vertex  $v$  is included in SPT , otherwise not. Array dist[ ] is used to store the shortest distance values of all vertices.

# Dijkstra's algorithm

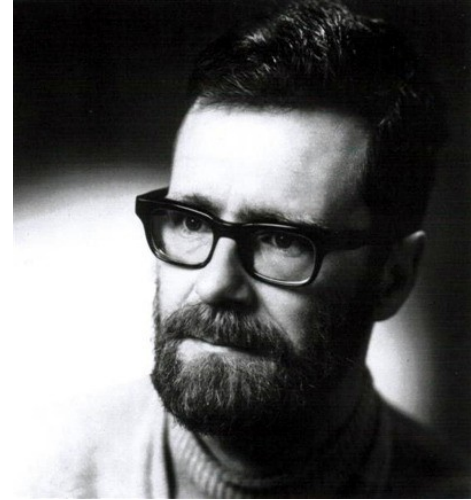
What is the shortest way to travel from Rotterdam to Groningen, in general: from given city to given city. It is the algorithm for the shortest path, which I designed in about twenty minutes. One morning I was shopping in Amsterdam with my young fiancée, and tired, we sat down on the café terrace to drink a cup of coffee and I was just thinking about whether I could do this, and I then designed the algorithm for the shortest path.

As I said, it was a twenty-minute invention.

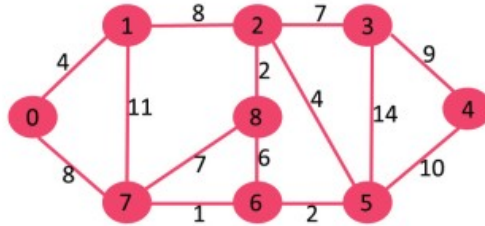
In fact, it was published in '59, three years later. The publication is still readable, it is, in fact, quite nice. One of the reasons that it is so nice was that I designed it without pencil and paper. I learned later that one of the advantages of designing without pencil and paper is that you are almost forced to avoid all avoidable complexities.

Eventually that algorithm became, to my great amazement, one of the cornerstones of my fame.

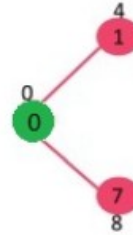
*Edsger Dijkstra, in an interview with Philip L. Frana,  
Communications of the ACM, 2001*



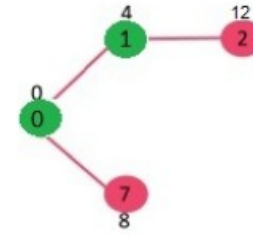
# Dijkstra's example



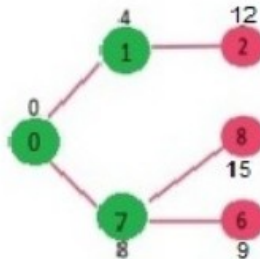
Initially,  $sptSet = \emptyset$ , the distance assigned to the source vertex 0 is 0 and the distance assigned to other vertices is  $\infty$ . The subgraph shows vertices and their distance values (only if they are finite). The vertices included in  $sptSet$  are shown in green.



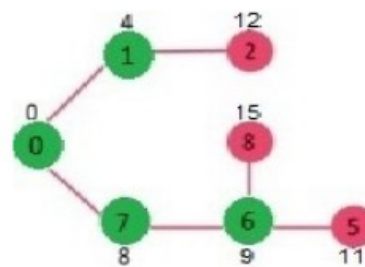
Pick the vertex  $\notin sptSet$  with minimum distance value. The vertex 0 is picked and included in  $sptSet$  so  $sptSet = \{0\}$ . Update distance values of 0's adjacent vertices. Adjacent vertices of 0 are 1 and 7. The distance values of 1 and 7 become 4 and 8 respectively.



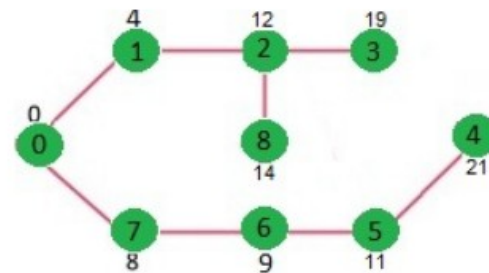
Pick the vertex  $\notin sptSet$  with minimum distance value. The vertex 1 is picked and included in  $sptSet$  so  $sptSet = \{0, 1\}$ . Update the distance values of 1's adjacent vertices. The distance value of vertex 2 becomes 12.



Pick the vertex  $\notin sptSet$  with minimum distance value. Vertex 7 is picked and included in  $sptSet$  so  $sptSet = \{0, 1, 7\}$ . Update the distance values of 7's adjacent vertices. The distance values of vertices 6 and 8 become 15 and 9 respectively.



Pick the vertex  $\notin sptSet$  with minimum distance value. Vertex 6 is picked and included in  $sptSet$  so  $sptSet = \{0, 1, 7, 6\}$ . Update the distance values of 6's adjacent vertices. The distance values of vertices 5 and 8 become 11 and 15 respectively.



Repeat the above steps until  $sptSet$  does include all vertices. Finally, the indicated SPT is obtained.

# Dijkstra's complexity

Dijkstra's time complexity is from  $O(|V| \log |V|)$  to  $O(|V|^2)$

Dijkstra's algorithm using a **Fibonacci heap** runs in amortized time  $O(|V| \log |V| + E)$ , which is the theoretical minimum for sparse graphs, although it is rarely used in practice due to high overhead.

The structure was named "Fibonacci" because the maximum degree (number of children) of a node in a heap of  $n$  elements is bounded by  $O(\log n)$ , and the sizes of the subtrees are closely related to Fibonacci numbers.

Starting numbers:  $F_0 = 0$  and  $F_1 = 1$

Recursive rule:  $F_n = F_{n-1} + F_{n-2}$  for  $n > 1$

The series: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, and onward.

A Fibonacci heap is a tree-based data structure that implements a priority queue with exceptionally fast theoretical performance. Its main difference from a standard binary heap is that most operations run in amortized  $O(1)$  time, thanks to a "lazy" approach (the structure defers the complex task of tree balancing until later).

The Fibonacci heap postpones tree restructuring until the minimum element is extracted, allowing for constant-time insertion and key-decrease operations.

**In practice**, however, a standard binary heap performs faster due to its simpler memory structure and lower constant factors in the code, despite having worse theoretical asymptotic complexity.

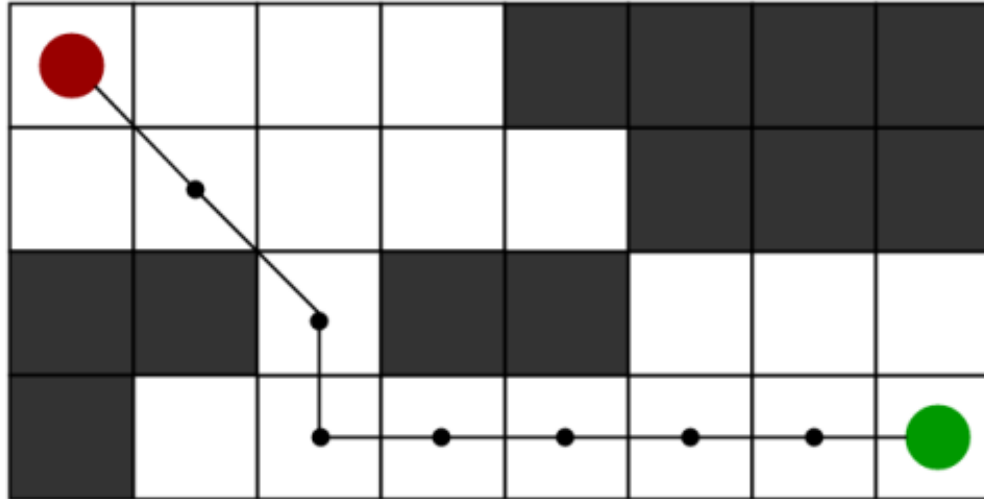
# A\* algorithm

**Problem:** given a weighted graph (with positive weights), a source vertex and a target vertex, find a shortest path from the source to the target.

**Main idea:** At each iteration, A\* determines how to extend the path basing on the cost of the current path from the source and an estimate of the cost required to extend the path to the target.

**Time complexity** is  $O(|E|)$ .

One gets Dijkstra's algorithm if the above-mentioned estimate is skipped. Consider a grid with obstacles that can be represented as a weighted graph (vertices=cells, weighted edges=shortest paths and their lengths).



In general, movements to different cells may have different cost.

# A\* example

We are given a square grid with obstacles, a source cell  $c_s$  and a target cell  $c_t$ . The aim is to reach  $c_t$  (if possible) from  $c_s$  as quickly as possible.

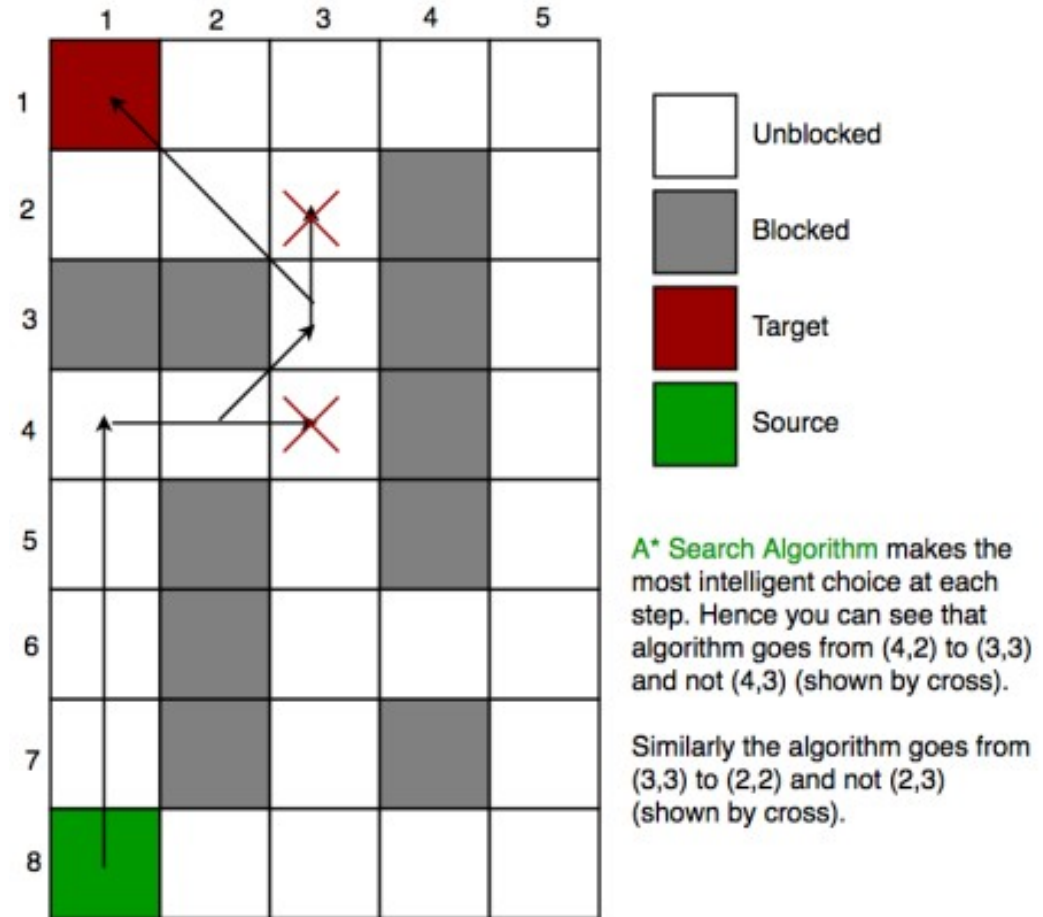
A\* extends the path according to the value of  $f(c_k) = g(c_k) + h(c_k)$ , where  $g$  measures the cost to move from  $c_s$  to  $c_k$  by the path generated and  $h$  measures the cost to move from  $c_k$  to  $c_t$ .

How to calculate  $h$ ?

**Exactly (time consuming):** pre-compute the distance between each pair of cells before running A\*.

**Approximately:** calculate the distances assuming no obstacles, e.g. Manhattan, Diagonal, Euclidean

A\* keeps all generated nodes in memory. In practice, this turns out to be the biggest drawback of the A\* search, leading to the development of memory-bounded heuristic searches, such as **Iterative deepening A\***, **memory-bounded A\***, and **Simplified Memory Bounded A\*** (SMA\*)



# Bellman-Ford algorithm

**Problem:** Given a weighted graph (possibly directed and with negative weights) and a source vertex  $s$ , find shortest paths from  $s$  to all vertices in the graph.

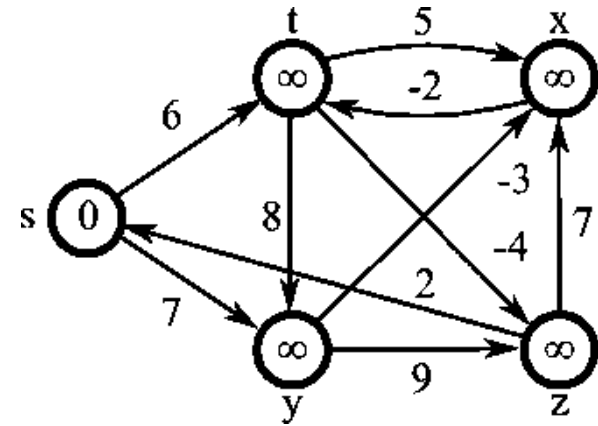
If a graph contains a negative cycle (i.e. a cycle whose edges sum to a negative value) that is reachable from  $s$ , then there is no shortest path: any path that has a point on the negative cycle can be made shorter by one more walk around the negative cycle. In such a case, Bellman-Ford can detect the negative cycle.

**Note:** Dijkstra does not generally work with negative weights. Bellman-Ford is simpler than Dijkstra but its time complexity is  $O(|V||E|)$ .

**Idea:** At  $i$ -th iteration, Bellman-Ford calculates the shortest paths which have at most  $i$  edges. As there is maximum  $|V| - 1$  edges in any simple path,  $i = 1, \dots, |V| - 1$ .

Assuming that there is no negative cycle, if we have calculated shortest paths with at most  $i$  edges, then an iteration over all edges guarantees to give shortest paths with at most  $(i + 1)$  edges.

To check if there is a negative cycle, make  $|V|$ -th iteration. If at least one of the shortest paths becomes shorter, there is such a cycle.



# Bellman-Ford algorithm

## Algorithm:

Initialize distance array  $\text{dist}[]$  for each vertex ' $v$ ' as  $\text{dist}[v] = \text{INFINITY}$ .

Assume any vertex (let's say ' $0$ ') as source and assign  $\text{dist} = 0$ .

Relax all the edges( $u, v, \text{weight}$ )  $N-1$  times as per the below condition:  $\text{dist}[v] = \text{minimum}(\text{dist}[v], \text{distance}[u] + \text{weight})$

Now, Relax all the edges one more time i.e. the  $N$ -th time and based on the below two cases we can detect the negative cycle:

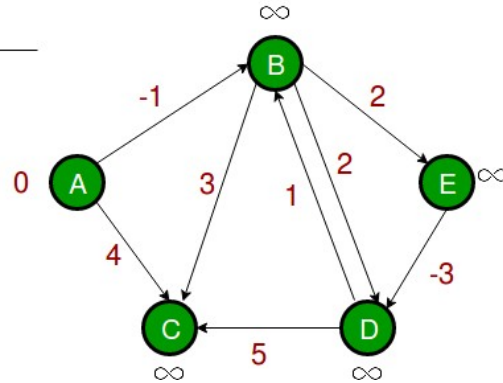
Case 1 (Negative cycle exists): For any edge( $u, v, \text{weight}$ ), if  $\text{dist}[u] + \text{weight} < \text{dist}[v]$

Case 2 (No Negative cycle) : case 1 fails for all the edges

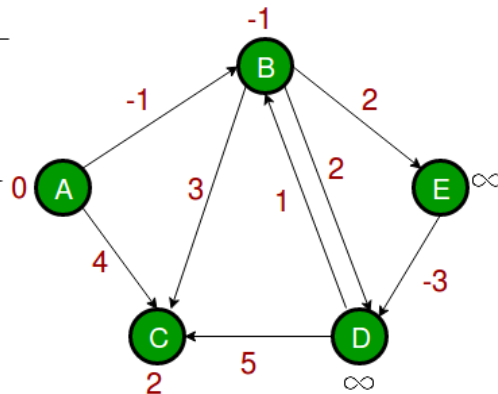
```
for  $v \in V$ 
  do  $d[v] \leftarrow +\infty$ 
 $d[s] \leftarrow 0$ 
for  $i \leftarrow 1$  to  $|V| - 1$ 
  do for  $(u, v) \in E$ 
    if  $d[v] > d[u] + w(u, v)$ 
      then  $d[v] \leftarrow d[u] + w(u, v)$ 
return  $d$ 
```

# Bellman-Ford algorithm

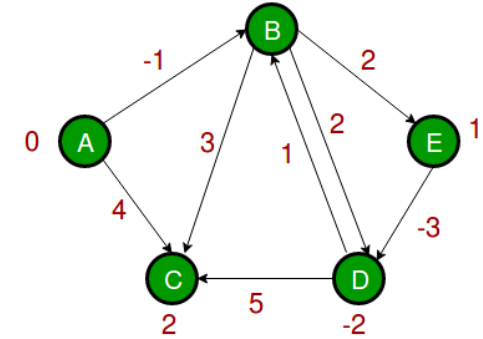
| A | B        | C        | D        | E        |
|---|----------|----------|----------|----------|
| 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ |



| A | B        | C        | D        | E        |
|---|----------|----------|----------|----------|
| 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 0 | -1       | $\infty$ | $\infty$ | $\infty$ |
| 0 | -1       | 4        | $\infty$ | $\infty$ |
| 0 | -1       | 2        | $\infty$ | $\infty$ |



| A | B        | C        | D        | E        |
|---|----------|----------|----------|----------|
| 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| 0 | -1       | $\infty$ | $\infty$ | $\infty$ |
| 0 | -1       | 4        | $\infty$ | $\infty$ |
| 0 | -1       | 2        | $\infty$ | $\infty$ |
| 0 | -1       | 2        | $\infty$ | 1        |
| 0 | -1       | 2        | 1        | 1        |
| 0 | -1       | 2        | -2       | 1        |



Negative weights may appear, for example, instead of increasing the cost of a path, we may gain a benefit by following a certain path.

Time complexity:  $O(V \cdot E)$



АНТОН КОВАНЦЕВ  
[pop.antonij@gmail.com](mailto:pop.antonij@gmail.com)  
[https://t.me/pop\\_antonij](https://t.me/pop_antonij)